

HO CHI MINH UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

STATISTICAL LEARNING



FINAL PROJECT REPORT

**Dearly: Secure Virtual Assistant for Elderly Care
with Speaker Verification and Identification**

LECTURER:

Lê Long Quốc

STUDENTS:

Lê Nguyên Thảo - 23127118

Đinh Đại Vũ - 23127144

Abstract

Dearly is a Vietnamese Android virtual assistant designed for elderly users and their caregivers. The application accepts recorded speech, transcribes the request, selects an assistant action, and returns both text and a Vietnamese spoken response. It provides a safe public “Ask Dearly” mode for time, date, and usage guidance before sign-in; personal information remains unavailable in that mode. For authenticated care tasks, Dearly combines automatic speech recognition (ASR), speaker verification (SV), speaker identification (SID), and server-side authorization. A fine-tuned ECAPA-TDNN embedding model supports both one-to-one verification and one-to-many identification [1, 2]; Whisper supports Vietnamese transcription [3]. The Android client shows listening state, a level waveform, and a transcript preview; its voice-activity gate discards silent/too-short recordings. For the hands-free medication use case, the elder first says that a named medicine has been taken, then confirms the medicine and scheduled time with a fresh spoken response. Dearly verifies that confirmation recording against the signed-in elder and issues a short-lived one-use grant only after a pass; replayed, too-short, or failed recordings leave medication state unchanged.

Contents

Abstract	2
Group Members and Contribution	5
1 Introduction and System Overview	6
1.1 Project Context and Objectives	6
1.2 Required Assistant Use Cases	6
1.3 System Architecture	7
1.4 Voice-Request Processing Flow	8
2 Dataset Preparation and Model Training	9
2.1 VIVOS Dataset and Speaker-Disjoint Splits	9
2.2 Audio Preprocessing and Speaker Enrollment	9
2.3 ECAPA-TDNN Model and Training Procedure	9
3 Experimental Results and System Integration	11
3.1 Evaluation Protocol and Experimental Results	11
3.2 Speaker Enrollment, Verification, and Identification Integration	11
3.2.1 Enrollment procedure and stored voice data	11
3.2.2 Verification and identification at runtime	12
3.3 Implementation Validation and Demonstration Evidence	13
3.4 Limitations and Conclusion	15
References	16

List of Figures

1	Dearly service architecture. The Android app does not call the database, Kafka, or AI service directly.	8
2	Hands-free medication confirmation. The named-dose request is followed by a fresh spoken confirmation; successful SV is required before the medication update.	13
3	Screenshot 1: completed speaker enrollment.	14
4	Screenshot 2: public general voice function.	14
5	Screenshots 3–6: hands-free SV-protected medication action, including success and failure.	14
6	Screenshot 7: SID-based personalized medication response.	14

List of Tables

1	Project group members.	5
2	Task distribution and contribution.	5
3	Dearly voice-assistant use cases and security controls.	7
4	Speaker-disjoint VIVOS split used by the completed experiment.	9
5	Fine-tuning configuration for the VIVOS ECAPA-TDNN experiment.	10
6	Held-out VIVOS evaluation results.	11
7	Information maintained for speaker verification and identification.	12

Group Members and Contribution

No.	Full name	Student ID	Email
1	Lê Nguyên Thảo	23127118	lnthao23@clc.fitus.edu.vn
2	Đinh Đại Vũ	23127144	ddvu23@clc.fitus.edu.vn

Table 1: Project group members.

No.	Work package	Responsible member	Contribution
1	Android client, voice-capture and enrollment user experience, caregiver/elder workflows, and integration testing.	Lê Nguyên Thảo	50%
2	ECAPA-TDNN training and evaluation, FastAPI and Go voice services, authorization design, experiment analysis, and report integration.	Đinh Đại Vũ	50%

Table 2: Task distribution and contribution.

1 Introduction and System Overview

1.1 Project Context and Objectives

Older adults benefit from hands-free access to time, medication information, and caregiver-supported routines. At the same time, commands that change health-related state must resist accidental or unauthorized execution. Dearly addresses this tension with the following objectives:

- provide voice-based interaction in Vietnamese through an accessible Android interface;
- provide a public general-information mode without a Dearly or Firebase session;
- require successful SV before a medication dose is marked as taken; and
- use SID for safe personalization, limited to a caller’s trusted caregiver–elder group.

1.2 Required Assistant Use Cases

Table 3 maps the required project behaviors to the implemented assistant functions. The public function intentionally exposes no personal care data. For the important medication action, the initial request only resolves the named unpaid dose. Dearly reads back its medicine name and scheduled time, then asks for a spoken “Đúng rồi” confirmation and requires SV on that fresh recording before any state change.

Category	Example function	Control
General	Public “Ask Dearly”: ask for the current time/date or how to use Dearly before sign-in.	No Firebase/Dearly session, no SV/SID, and no profile, medication, or contact data.
Important	Say “Tôi đã uống Amlodipine rồi”, then say “Đúng rồi” after Dearly reads back the medicine and scheduled time.	Resolve today’s unpaid dose; require successful SV of the confirmation recording and a short-lived, one-use voice grant.
Personalized	Ask “Hôm nay tôi cần uống thuốc gì?” and receive the identified person’s name and schedule.	One-to-many SID over only the caller and directly linked caregiver/elder profiles; backend re-checks the relationship before loading data.

Table 3: Dearly voice-assistant use cases and security controls.

1.3 System Architecture

Figure 1 shows the production design. Android is the only public client. It records audio, sends it to the Go API, displays the response, and uses Android’s local `vi-VN` TextToSpeech engine for output. The Go API authenticates sessions, manages enrollment and medication state, and calls the private FastAPI service. The AI service runs ASR, ECAPA embedding extraction, SID, SV, and constrained LLM/local intent analysis. PostgreSQL is authoritative for user, relationship, medication, enrollment, and grant data; Kafka delivers domain events to the notification worker.

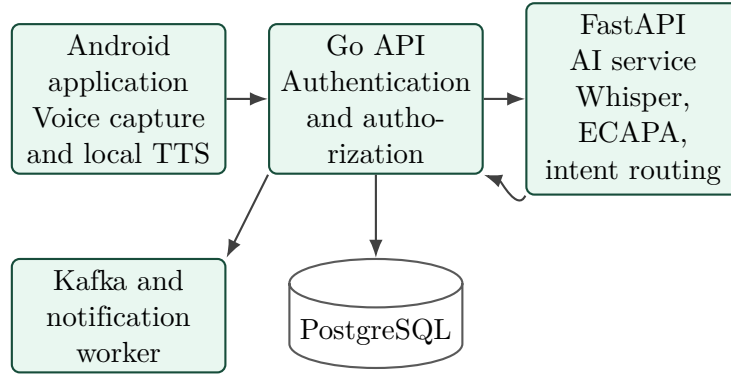


Figure 1: Dearly service architecture. The Android app does not call the database, Kafka, or AI service directly.

1.4 Voice-Request Processing Flow

For a medication confirmation, the read-back includes both the matched medicine and its scheduled time before Dearly accepts a fresh “Đúng rồi” response. For public queries, Android posts audio to a separate unauthenticated endpoint. The API permits only `ASK_TIME`, `ASK_DATE`, greeting, and usage guidance; it never loads enrollment, medication, contact, or relationship records. For authenticated queries, the AI service transcribes the recording with Whisper, identifies a speaker when trusted profiles are supplied, and classifies the request. The API enriches `ASK_TIME`, `ASK_DATE`, and `CHECK_MEDICATIONS` from local time or PostgreSQL. A spoken `MARK_TAKEN` request includes the medicine name: Android finds the matching unpaid dose, asks the elder to say “Đúng rồi,” and verifies that confirmation recording against the signed-in elder. Only a successful SV result creates a grant; therefore ASR, intent classification, and SID alone never authorize a medication update.

2 Dataset Preparation and Model Training

2.1 VIVOS Dataset and Speaker-Disjoint Splits

The speaker-recognition experiment uses VIVOS, a Vietnamese speech corpus released for academic use under the CC BY-NC-SA 4.0 license [4]. VIVOS is appropriate for this project because the final assistant operates in Vietnamese. The completed run places every speaker in exactly one partition, avoiding identity leakage between training, checkpoint selection, and final evaluation.

Split	Speakers	Utterances	Purpose
Training	41	10,310	Fine-tune the ECAPA embedding model and AAM-Softmax head.
Validation	5	1,350	Select the checkpoint with the best verification validation loss.
Test	19	760	Hold-out speaker-verification and speaker-identification evaluation only.

Table 4: Speaker-disjoint VIVOS split used by the completed experiment.

2.2 Audio Preprocessing and Speaker Enrollment

The training pipeline builds manifests containing utterance identifiers, absolute audio paths, speaker identifiers, and durations. At load time, recordings are converted to mono, resampled to 16 kHz when necessary, and transformed into fixed three-second examples. Training crops use a random start point; validation uses a deterministic center crop. Short recordings are zero-padded. This makes batches comparable while preserving the original speaker-disjoint assignment.

The Android enrollment workflow requests five prompted phrases. A caregiver may assist an elder, but each registered person records their own voice. The prompts cover medication, calling a relative, medication confirmation, an appointment reminder, and an identity statement. The client stores each recording only as a temporary file and posts it with a `phrase_index` from 0 to 4. FastAPI converts the recording to a 192-dimensional ECAPA embedding, deletes its temporary audio file, and returns only the embedding. PostgreSQL stores each phrase embedding as a serialized `float32 BYTEA` vector associated with the authenticated `user_id`. When all five phrases are present, the backend computes the element-wise average and stores it at `phrase_index=-1`; this is the profile used at runtime. Raw enrollment audio is not persisted. Re-recording a phrase replaces only that phrase’s vector, and reset enrollment deletes the user’s enrollment rows. The average profile supports both one-to-one SV and one-to-many SID; the latter receives only average profiles belonging to the caller and directly linked caregiver/elder accounts.

2.3 ECAPA-TDNN Model and Training Procedure

ECAPA-TDNN extends TDNN-based speaker representation learning with channel attention, multi-layer feature aggregation, and attentive statistics pooling [1]. Dearly starts from the SpeechBrain pretrained `speechbrain/spkrec-ecapa-voxceleb` source and fine-tunes

the embedding model on VIVOS [2]. The deployed runtime produces a finite 192-dimensional embedding, compares profiles by cosine similarity, and applies separate configurable SV and SID thresholds.

Setting	Completed experiment value
Compute environment	Google Colab Tesla T4 GPU
Final training duration	20 epochs, initialized from the original pretrained source
Batch size and audio input	32 examples; three-second mono 16 kHz crops
Objective	AAM-Softmax, margin 0.2 and scale 30
Optimizer and schedule	AdamW; cosine learning-rate decay from 10^{-4} to 10^{-6}
Backbone schedule	Frozen in epochs 1–5 and unfrozen in epochs 6–20
Checkpoint selection	Minimum validation loss: 0.2644617259502411

Table 5: Fine-tuning configuration for the VIVOS ECAPA-TDNN experiment.

The experimental sequence first evaluated the untouched VoxCeleb-pretrained baseline, then ran a one-epoch smoke experiment, and finally ran the independent 20-epoch training procedure. The final checkpoint is loaded by the production ECAPA wrapper rather than by a separate evaluation-only implementation. This checks that the artifact evaluated for the report can generate the same type of runtime embedding used for SV and SID.

3 Experimental Results and System Integration

3.1 Evaluation Protocol and Experimental Results

Speaker verification is evaluated with 10,000 deterministic balanced trials on held-out VIVOS speakers: 5,000 genuine trials and 5,000 impostor trials. Given two embedding vectors a and b , the runtime score is cosine similarity:

$$\text{score}(a, b) = \frac{a \cdot b}{\|a\|_2 \|b\|_2}.$$

The report uses equal error rate (EER), the point where false reject and false accept rates are equal, and minimum detection cost function (minDCF). SID Top-1 accuracy is computed by averaging five enrollment embeddings for each test speaker, then assigning each remaining query recording to the highest-scoring profile.

System	EER	minDCF	SID Top-1
Untouched VoxCeleb ECAPA baseline	4.40%	0.002978	100.00%
One-epoch smoke model	1.58%	0.001570	100.00%
VIVOS fine-tuned ECAPA	1.28%	0.001628	100.00%

Table 6: Held-out VIVOS evaluation results.

Fine-tuning lowers EER by 3.12 percentage points, or 70.9% relative to the baseline. The experiment’s EER operating threshold is 0.34559793992808. It is not automatically deployed: the current application threshold remains 0.80 and requires calibration on representative elderly-user recordings before it can be changed. The test-set result therefore demonstrates Vietnamese-domain performance, not a clinical or elderly-population validation.

3.2 Speaker Enrollment, Verification, and Identification Integration

3.2.1 Enrollment procedure and stored voice data

Enrollment starts after the new user has created a Dearly account. In *Settings – Voice enrollment*, the user records five displayed Vietnamese prompts with visible 0/5–5/5 progress. A caregiver may help an elder operate the device, but the enrolled speaker must say each prompt personally. Android’s amplitude-based VAD requires at least 0.8 seconds of sustained activity; silent or too-short recordings are deleted locally and the elder is asked to record that same phrase again. The Android client sends only an accepted temporary audio file and its `phrase_index` 0–4 to the authenticated `/api/v1/voice/enroll` endpoint. The API obtains the account identifier from the JWT rather than trusting a client-provided user identifier.

The Go API forwards the recording to FastAPI. The AI service validates the audio, performs a second intelligible-speech check with ASR before extracting an ECAPA-TDNN embedding, and deletes the temporary server-side file in a `finally` block. Thus, a direct API client cannot enroll a silent/unintelligible recording merely by bypassing Android’s VAD. The Android

client also deletes its temporary recording after the upload. Consequently, the enrollment database contains embeddings, not raw voice recordings. Each of the five phrase vectors is stored first; after phrase 5, the backend computes an element-wise mean and stores the resulting runtime profile at `phrase_index=-1`. Re-recording a phrase upserts that phrase’s row and recomputes the average when all five inputs are available. A reset deletes all enrollment rows for that user.

Stored item	Purpose and retention
<code>user_id</code>	UUID foreign key to the registered user; it binds an embedding to its owner and is deleted with that user.
<code>phrase_index 0–4</code>	Identifies one of the five prompted recordings; the unique user/index pair allows a phrase to be replaced safely.
<code>embedding_vector</code>	Serialized 192-dimensional <code>float32</code> ECAPA-TDNN vector stored as <code>BYTEA</code> ; no raw recording is needed for later SV or SID.
<code>phrase_index=-1</code>	Element-wise average of the five phrase embeddings; this is the only profile supplied to runtime verification and identification.
<code>created_at</code>	Records the most recent enrollment or replacement time for management and audit purposes.

Table 7: Information maintained for speaker verification and identification.

3.2.2 Verification and identification at runtime

For SV, ECAPA extracts an embedding from fresh speech and the AI service computes cosine similarity against the signed-in user’s average profile. The deployment threshold is currently 0.80. A pass produces a random grant bound to the user and protected intent; its SHA-256 hash is stored, it expires after two minutes, and the medication endpoint consumes it exactly once. The API records protected verification attempts, rejects a replay of a previously successful raw recording for 24 hours, and pauses verification for 15 minutes after five failed attempts. The named-dose/time read-back followed by a new spoken confirmation is an interaction check; exact raw-audio replay is blocked, but a dedicated anti-spoof/liveness model is not yet part of the system and re-encoded playback remains a stated limitation. For SID, the backend supplies only average profiles for the caller and directly linked caregiver/elder accounts. FastAPI returns the highest-scoring identity only when it meets the SID threshold. The Go API then independently re-checks that the returned identity is still in that trusted relationship group before it loads a name or medication schedule.

Figure 2 describes the hands-free protected medication flow. The elder does not need to tap a particular dose: they can say, for example, “Tôi đã uống Amlodipine rồi.” Dearly extracts the medicine name and finds today’s matching unpaid dose, but does not update it yet. Dearly reads back the medicine and scheduled time, for example, “Bác xác nhận đã uống Amlodipine lúc 08:30 phải không?”, then asks the elder to say “Đúng rồi.” SV runs on this fresh confirmation recording before the grant is issued. A silent, too-short, replayed, or failed confirmation leaves the dose unchanged and asks the elder to retry.

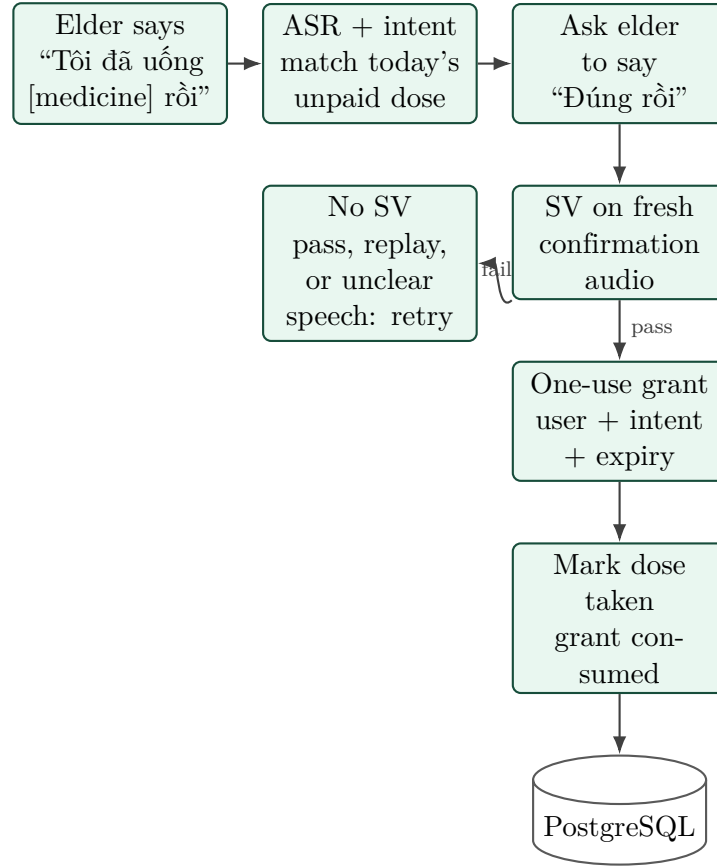


Figure 2: Hands-free medication confirmation. The named-dose request is followed by a fresh spoken confirmation; successful SV is required before the medication update.

3.3 Implementation Validation and Demonstration Evidence

SID personalization beyond the name. After a SID match is accepted and its caregiver/elder relationship is re-checked, the backend loads preferences only for that identified person: gentle or direct reminder wording, Vietnamese TTS speed, an optional preferred contact, and daily-schedule inclusion. The response contains these preferences; Android applies the selected rate to that TTS response and displays a small personalization label. Public requests and untrusted identities cannot read them.

The completed engineering checks include AI-service linting and focused tests, Docker Compose configuration validation, backend unit tests for voice grants, replay hashing, and general responses, protected-route authorization checks, and successful Android debug build and unit-test runs. The public voice route is reachable without a token, whereas the medication update route rejects unauthenticated requests or requests without a valid voice grant. These checks validate the contract and the compiled client; they do not replace a recorded demonstration with real enrolled voices.

The following captioned placeholders reserve the exact visual evidence required for the final submission. They must be replaced with real screenshots or video frames before the report is submitted.

Pending live demonstration evidence

Elder Settings → Voice Security, showing the enrollment dialog with all 5/5 phrases saved.

Replace this framed placeholder with the final device screenshot before submission.

Figure 3: Screenshot 1: completed speaker enrollment.

Pending live demonstration evidence

Public Dearly chat before sign-in: ask “Bây giờ là mấy giờ?” and show the completed Vietnamese answer.

Replace this framed placeholder with the final device screenshot before submission.

Figure 4: Screenshot 2: public general voice function.

Pending live demonstration evidence

Create a four-panel sequence: (1) Elder Medication with a dose marked “Chưa uống”; (2) after saying “Tôi đã uống Amlodipine rồi”, Dearly asks for “Đúng rồi” with the correct medicine and time; (3) a successful “Đúng rồi” verification marks that dose “Đã xong” or “Đã uống”; and (4) a different or unregistered voice fails verification while another unpaid dose remains “Chưa uống”.

Replace this framed placeholder with the final device screenshot before submission.

Figure 5: Screenshots 3–6: hands-free SV-protected medication action, including success and failure.

Pending live demonstration evidence

Create a two-panel comparison: for each of two linked and enrolled users, ask “Hôm nay tôi cần uống thuốc gì?” and show that person’s recognized name and own medication schedule.

Replace this framed placeholder with the final device screenshot before submission.

Figure 6: Screenshot 7: SID-based personalized medication response.

3.4 Limitations and Conclusion

Dearly provides a reportable end-to-end design: VIVOS fine-tuning improves held-out verification performance, ECAPA embeddings drive both SV and SID, and a server-side grant prevents ASR, intent classification, or SID alone from authorizing a protected medication update. The public assistant demonstrates a genuinely non-authenticated function without exposing private information. The remaining limitation is final evidence collection: the submission should replace every placeholder with real-device screenshots or video frames for enrollment, public interaction, successful hands-free medication confirmation, failed verification, and two-person SID personalization. The final package must include the source code, report, and a Drive-link text file for large dataset/model artifacts when needed. The ZIP filename for this team should use both student IDs: `23127118_23127144.zip`.

References

- [1] Brecht Desplanques, Jenthe Thienpondt, and Kris Demuynck. Ecapa-tdnn: Emphasized channel attention, propagation and aggregation in tdnn based speaker verification. In *Proceedings of Interspeech 2020*, pages 3830–3834, 2020.
- [2] Mirco Ravanelli, Titouan Parcollet, Peter Plantinga, et al. Speechbrain: A general-purpose speech toolkit. *arXiv preprint arXiv:2106.04624*, 2021.
- [3] Alec Radford, Jong Wook Kim, Tao Xu, et al. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.
- [4] Hieu-Thi Luong and Hai-Quan Vu. A non-expert kaldi recipe for vietnamese speech recognition system. In *Proceedings of the 4th Workshop on Linked Data in Linguistics*, 2016. VIVOS Vietnamese Speech Corpus, CC BY-NC-SA 4.0.
- [5] OpenAI. Gpt-4o system card, 2024. Accessed September 2026.